Hadoop Storgare - HDFS

Vũ Tuyết Trinh
SoICT - HUST

1

Outline

- File Systems
- Distributed File Systems
- Design Issues in Distributed File Systems
- HDFS
  - an overview
  - Architecture
  - Namespace
  - Data replication
  - HDFS data format

2

File systems



NTFS

hadoop HDFS

Ext4 File System

NFS NETWORK FILE SYSTEM

PVFS

The Google File System

By Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung
(Presented at SOSP 2003)

3

Overview

- Filenames
  - File Identity
- Directories (folders)
  - Group of files in separate collections
- Metadata
  - Creation time, last access time, last modification time
  - Security information (Owner, Group owner)
  - Mapping file to its physical location of file (e.g. location in storage devices)
- Computer file
  - A resource for storing information
  - Durable, remained available for access
  - Data: sequences of bits
- File system
  - Control how computer file are stored and retrieved
  - Main operators: READ, WRITE (offset, size), CREATE, DELETE

4

Local vs. distributed file systems





NFS
NETWORK FILE SYSTEM

The Google File System

By Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung
(Presented at SOSP 2003)

PVFS

5

Distributed file system

- File system
  - Abstraction of storage devices
- Distributed file system
  - Available to remote processes in distributed systems
- Benefits
  - File sharing
  - Uniform view of system from different clients
  - Centralized administration

6

Goals: Network (Access) Transparency

- Network (Access) Transparency
  - Users should be able to access files over a network as easily as if the files were stored locally.
  - Users should not have to know the physical location of a file to access it.
- Transparency can be addressed through naming and file mounting mechanisms
  - Location Transparency: file name doesn't specify physical location
  - Location Independence: files can be moved to new physical location, no need to change references to them. (A name is independent of its addresses)
  - Location independence → location transparency, but the reverse is not necessarily true.

7

Goals: Availability

- Availability: files should be easily and quickly accessible.
- The number of users, system failures, or other consequences of distribution shouldn't compromise the availability.
- Addressed mainly through replication.

8

Architectures

- Client-Server
  - Sun Microsystem Network File System (NFS), Google File System (GFS)
  - Architecture
    - One or more machines (file servers) manage the file system.
    - Files are stored on disks at the servers
    - Requests for file operations are made from clients to the servers.
    - Client-server systems centralize storage and management; P2P systems decentralize it.
- P2P (Symmetric)
  - Fully decentralized; based on peer-to-peer technology
  - e.g., Ivy (uses a Chord DHT approach)

9

Design issues in distributed file systems

- Naming and name resolution
- Semantics of file sharing
- Caching
- Replication

10

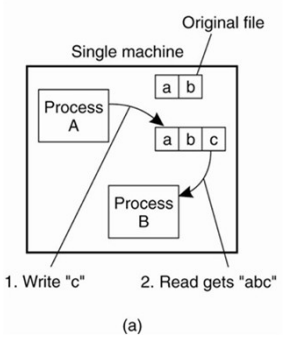
Naming and name resolution

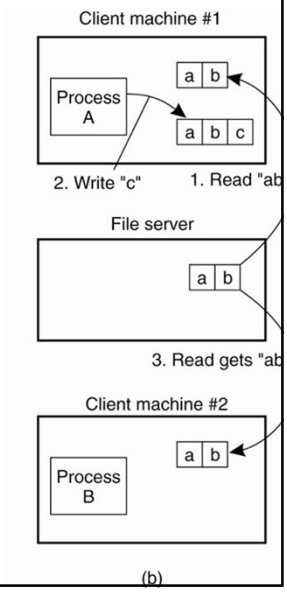
- A name space -- collection of names
- Name resolution -- mapping a name to an object
- 3 traditional ways
  - Concatenate the host name to the names of files stored on that host
  - Mount remote directories onto local directories
  - Provide a single global directory

11

File Sharing Semantics (1/2)

- **Problem**: When dealing with distributed file systems, we need to consider the ordering of concurrent read/write operations and expected semantics (=consistency).





12

File Sharing Semantics (2/2)

- Assume open; reads/writes; close
  - **UNIX semantics:** value read is the value stored by last write
    Writes to an open file are visible immediately to others that have this file opened at the same time. Easy to implement if one server and no cache.
  - **Session semantics:**
    Writes to an open file by a user is not visible immediately by other users that have files opened already.
    Once a file is closed, the changes made by it are visible by sessions started later.
  - **Immutable-Shared-Files semantics:**
    A sharable file cannot be modified.
    File names cannot be reused and its contents may not be altered.
    Simple to implement.
  - **Transactions:** All changes have all-or-nothing property.
    W1,R1,R2,W2 not allowed where P1 = W1;W2 and P2 = R1;R2

13

Caching

- Server caching: in main memory
  - cache management issue, how much to cache, replacement strategy
  - still slow due to network delay
  - Used in high-performance web-search engine servers
- Client caching in main memory
  - can be used by diskless workstation
  - faster to access from main memory than disk
  - compete with the virtual memory system for physical memory space
- Client-cache on a local disk
  - large files can be cached
  - the virtual memory management is simpler
  - a workstation can function even when it is disconnected from the network

14

Caching tradeoffs

- Reduces remote accesses => reduces network traffic and server load
- Total network overhead is lower for big chunks of data (caching) than a series of responses to specific requests.
- Disk access can be optimized better for large requests than random disk blocks
- Cache-consistency problem is the major drawback. If there are frequent writes, overhead due to the consistency problem is significant.

15

Replication

- File data is replicated to multiple storage servers
- Goals
  - Increase reliability
  - improve availability
  - balance the servers workload
- How to make replication transparent?
- How to keep the replicas consistent?
  - a replica is not updated due to its server failure
  - network partitioned

16

HDFS – an Overview

- Provides inexpensive and reliable storage for massive amounts of data
- Designed for
  - Big files (100 MB to several TBs file sizes)
  - Write once, read many times (Appending only)
  - Running on commodity hardware
- Hierarchical UNIX style file systems
  - (e.g., /hust/soict/hello.txt)
  - UNIX style file ownership and permissions

17

17

HDFS main design principles

- I/O pattern
  - Append only → reduce synchronization
- Data distribution
  - File is splitted in big chunks (64 MB)

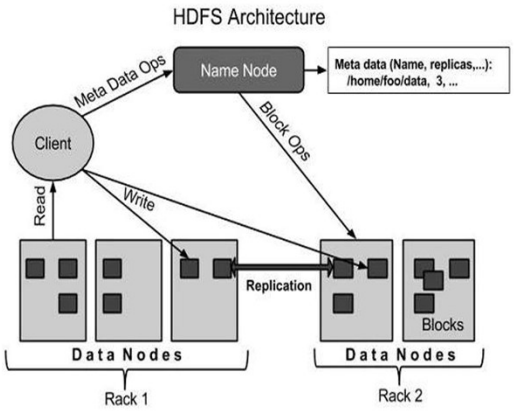
  → reduce metadata size

  → reduce network communication
- Data replication
  - Each chunk is usually replicated in 3 different nodes
- Fault tolerance
  - Data node: re-replication
  - Name node
    - Secondary Namenode
    - Standby, Active Namenodes

18

HDFS Architecture

- Master/slave architecture
- HDFS master: Namenode
  - Manage namespace and metadata
  - Monitor Datanode
- HDFS slaves: Datanodes
  - Handle read/write the actual data {chunks}
  - Chunks are local files in the local file systems



19

19

Functions of a Namenode

- Manages File System Namespace
  - Maps a file name to a set of blocks
  - Maps a block to the Datanodes where it resides
- Cluster Configuration Management
- Replication Engine for Blocks

20

Namenode metadata

- Metadata in memory
  - The entire metadata is in main memory
  - No demand paging of metadata
- Types of metadata
  - List of files
  - List of Blocks for each file
  - List of Datanodes for each block
  - File attributes, e.g. creation time, replication factor
- A Transaction Log
  - Records file creations, file deletions etc

21

Datanode

- A Block Server
  - Stores data in the local file system (e.g. ext3)
  - Stores metadata of a block (e.g. CRC)
  - Serves data and metadata to Clients
- Block Report
  - Periodically sends a report of all existing blocks to the Namenode
- Facilitates Pipelining of Data
  - Forwards data to other specified Datanodes
- Heartbeat
  - Datanodes send heartbeat to the Namenode
    - Once every 3 seconds
  - Namenode uses heartbeats to detect Datanode failure

22

Data replication

- Chunk placement
  - Current Strategy
    - One replica on local node
    - Second replica on a remote rack
    - Third replica on same remote rack
    - Additional replicas are randomly placed
  - Clients read from nearest replicas
- Namenode detects Datanode failures
  - Chooses new Datanodes for new replicas
  - Balances disk usage
  - Balances communication traffic to Datanodes

23

Data rebalance

- Goal: % disk full on Datanodes should be similar
  - Usually run when new Datanodes are added
  - Cluster is online when Rebalancer is active
  - Rebalancer is throttled to avoid network congestion
  - Command line tool

24

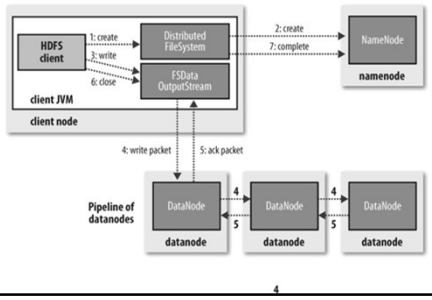
Data correctness

- Use Checksums to validate data
  - Use CRC32
- File Creation
  - Client computes checksum per 512 bytes
  - Datanode stores the checksum
- File access
  - Client retrieves the data and checksum from Datanode
  - If Validation fails, Client tries other replicas

25

Data pipelining

- Client retrieves a list of Datanodes on which to place replicas of a block
- Client writes block to the first Datanode
- The first Datanode forwards the data to the next node in the Pipeline
- When all replicas are written, the Client moves on to write the next block in file



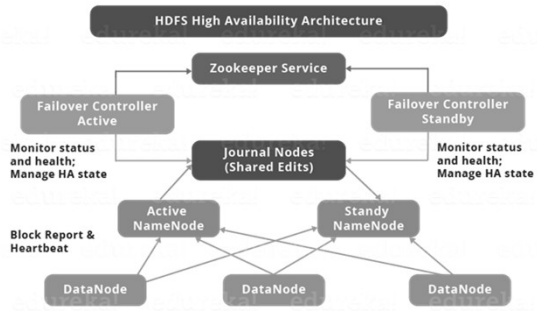
26

Secondary Name node

- Namenode is a single point of failure
- Secondary Namenode
  - Checkpointing latest copy of the FsImage and the Transaction Log files.
    - Copies FsImage and Transaction Log from Namenode to a temporary directory
- When Namenode restarted
  - Merges FSImage and Transaction Log into a new FSImage in temporary directory
  - Uploads new FSImage to the Namenode
    - Transaction Log on Namenode is purged

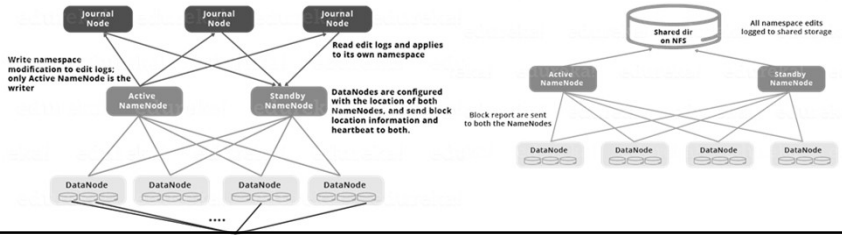
27

Namenode high availability (HA)



Quorum Journal Nodes

Shared Storage



28

HDFS command-line interface

List Files	
hdfs dfs -ls /	List all the files/directories for the given hdfs destination path.
hdfs dfs -ls -d /hadoop	Directories are listed as plain files. In this case, this command will list the details of hadoop folder.
hdfs dfs -ls -h /data	Format file sizes in a human-readable fashion (eg 64.0m instead of 67108864).
hdfs dfs -ls -R /hadoop	Recursively list all files in hadoop directory and all subdirectories in hadoop directory.
hdfs dfs -ls /hadoop/dat*	List all the files matching the pattern. In this case, it will list all the files inside hadoop directory which starts with 'dat'.

Read/Write Files	
hdfs dfs -text /hadoop/derby.log	HDFS Command that takes a source file and outputs the file in text format on the terminal. The allowed formats are zip and TextRecordInputStream.
hdfs dfs -cat /hadoop/test	This command will display the content of the HDFS file test on your stdout .
hdfs dfs -appendToFile /home/ubuntu/test1 /hadoop/text2	Appends the content of a local file test1 to a hdfs file test2.

29

Upload, download files

Upload/Download Files	
hdfs dfs -put /home/ubuntu/sample /hadoop	Copies the file from local file system to HDFS.
hdfs dfs -put -f /home/ubuntu/sample /hadoop	Copies the file from local file system to HDFS, and in case the local already exits in the given destination path, using -f option with put command will overwrite it.
hdfs dfs -put -l /home/ubuntu/sample /hadoop	Copies the file from local file system to HDFS. Allow DataNode to lazily persist the file to disk. Forces replication factor of 1.
hdfs dfs -put -p /home/ubuntu/sample /hadoop	Copies the file from local file system to HDFS. Passing -p preserves access and modification times, ownership and the mode.
hdfs dfs -get /newfile /home/ubuntu/	Copies the file from HDFS to local file system.
hdfs dfs -get -p /newfile /home/ubuntu/	Copies the file from HDFS to local file system. Passing -p preserves access and modification times, ownership and the mode.
hdfs dfs -get /hadoop/*.txt /home/ubuntu/	Copies all the files matching the pattern from local file system to HDFS.
hdfs dfs -copyFromLocal /home/ubuntu/sample /hadoop	Works similarly to the put command, except that the source is restricted to a local file reference.
hdfs dfs -copyToLocal /newfile /home/ubuntu/	Works similarly to the put command, except that the destination is restricted to a local file reference.
hdfs dfs -moveFromLocal /home/ubuntu/sample /hadoop	Works similarly to the put command, except that the source is deleted after it's copied.

30

File management

File Management	
hdfs dfs -cp /hadoop/file1 /hadoop1	Copies file from source to destination on HDFS. In this case, copying file1 from hadoop directory to hadoop1 directory.
hdfs dfs -cp -p /hadoop/file1 /hadoop1	Copies file from source to destination on HDFS. Passing -p preserves access and modification times, ownership and the mode.
hdfs dfs -cp -f /hadoop/file1 /hadoop1	Copies file from source to destination on HDFS. Passing -f overwrites the destination if it already exists.
hdfs dfs -mv /hadoop/file1 /hadoop1	Move files that match the specified file pattern <src> to a destination <dst>. When moving multiple files, the destination must be a directory.
hdfs dfs -rm /hadoop/file1	Deletes the file (sends it to the trash).
hdfs dfs -rm -r /hadoop hdfs dfs -rm -R /hadoop hdfs dfs -rmr /hadoop	Deletes the directory and any content under it recursively.
hdfs dfs -rm -skipTrash /hadoop	The -skipTrash option will bypass trash, if enabled, and delete the specified file(s) immediately.
hdfs dfs -rm -f /hadoop	If the file does not exist, do not display a diagnostic message or modify the exit status to reflect an error.
hdfs dfs -rmdir /hadoop1	Delete a directory.
hdfs dfs -mkdir /hadoop2	Create a directory in specified HDFS location.
hdfs dfs -mkdir -f /hadoop2	Create a directory in specified HDFS location. This command does not fail even if the directory already exists.
hdfs dfs -touchz /hadoop3	Creates a file of zero length at <path> with current time as the timestamp of that <path>.

31

Ownership and validation

Ownership and Validation	
hdfs dfs -checksum /hadoop/file1	Dump checksum information for files that match the file pattern <src> to stdout.
hdfs dfs -chmod 755 /hadoop/file1	Changes permissions of the file.
hdfs dfs -chmod -R 755 /hadoop	Changes permissions of the files recursively.
hdfs dfs -chown ubuntu:ubuntu /hadoop	Changes owner of the file. 1st ubuntu in the command is owner and 2nd one is group.
hdfs dfs -chown -R ubuntu:ubuntu /hadoop	Changes owner of the files recursively.
hdfs dfs -chgrp ubuntu /hadoop	Changes group association of the file.
hdfs dfs -chgrp -R ubuntu /hadoop	Changes group association of the files recursively.
Filesystem	
hdfs dfs -df /hadoop	Shows the capacity, free and used space of the filesystem.
hdfs dfs -df -h /hadoop	Shows the capacity, free and used space of the filesystem. -h parameter Formats the sizes of files in a human-readable fashion.
hdfs dfs -du /hadoop/file	Show the amount of space, in bytes, used by the files that match the specified file pattern.
hdfs dfs -du -s /hadoop/file	Rather than showing the size of each individual file that matches the pattern, shows the total (summary) size.
hdfs dfs -du -h /hadoop/file	Show the amount of space, in bytes, used by the files that match the specified file pattern. Formats the sizes of files in a human-readable fashion.

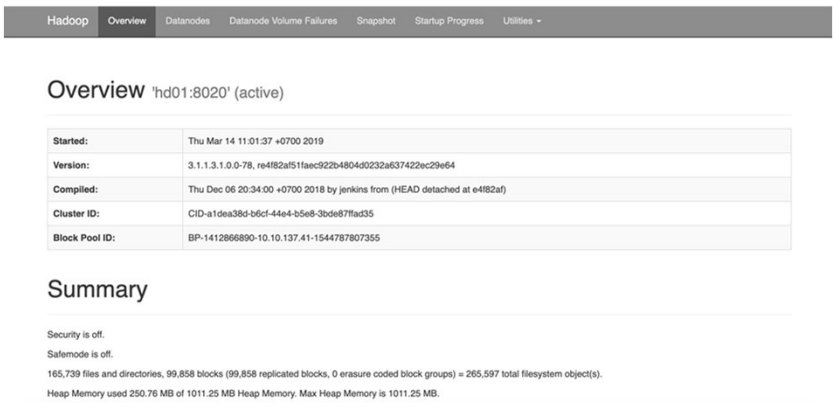
32

Administration

Administration	
hdfs balancer -threshold 30	Runs a cluster balancing utility. Percentage of disk capacity. This overwrites the default threshold.
hadoop version	To check the vesrion of Hadoop.
hdfs fsck /	It checks the health of the Hadoop file system.
hdfs dfsadmin -safemode leave	The command to turn off the safemode of NameNode.
hdfs dfsadmin -refreshNodes	Re-read the hosts and exclude files to update the set of Datanodes that are allowed to connect to the Namenode and those that should be decommissioned or recommissioned.
hdfs namenode -format	Formats the NameNode.

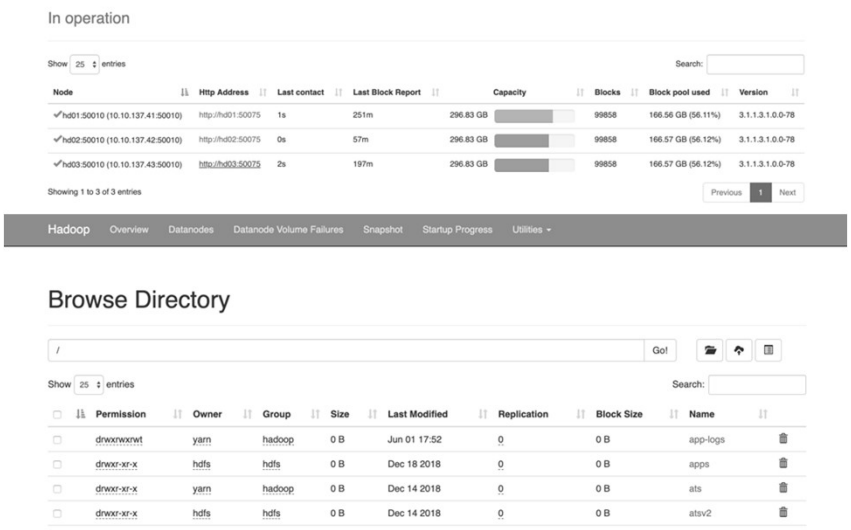
33

HDFS Name node UI



34

HDFS Name node UI (2)



35

Other HDFS interfaces

- Java API
- Thrift API
- Fuse
- WebDAV

36

HDFS data format

- Text
- Sequence file
- Avro
- Parquet
- Optimized Row Columnar (ORC)

37

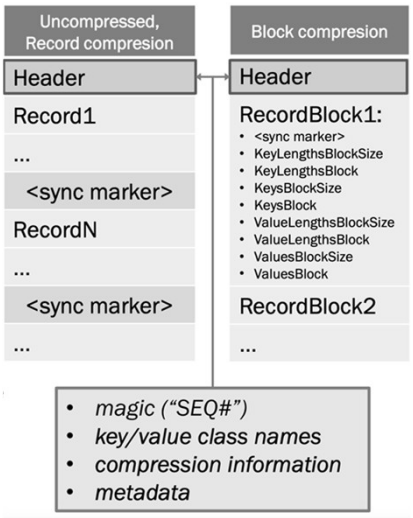
Text file

- CSV, TSV, Json records
- Convenient format to use to exchange between applications or scripts
- Human readable and parsable
- Do not support block compression
- Not as efficient to query
- Good for the beginning, but not good enough for real life.

38

Sequence file

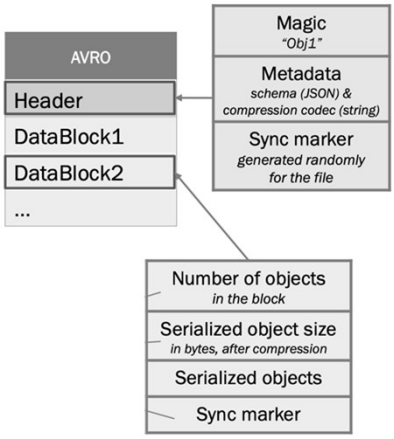
- Provides a persistent data structure for binary key-value pairs
- Commonly used to transfer data between Map Reduce jobs
- Can be used as an archive to pack small files in Hadoop
- Row-based
- Compression
- Splittable
  - Support splitting even when the data is compressed



39

Avro

- Row based
- Supports (object) compression and splitting
- Flexible data scheme
  - Schema (JSON) included to the file
- Data types
  - primitive: null, boolean, int, long, ...
  - complex: records, arrays, maps, ...
- Binary and JSON data serialization
- Data corruption detection



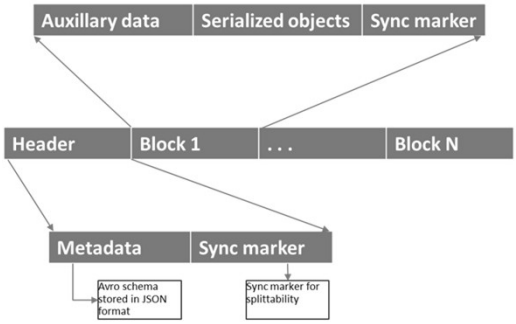
40

Avro – File structure and example

Sample AVRO schema in JSON format

```
{
  "type" : "record",
  "name" : "tweets",
  "fields" : [ {
    "name" : "username",
    "type" : "string",
}, {
    "name" : "tweet",
    "type" : "string",
}, {
    "name" : "timestamp",
    "type" : "long",
} ],
  "doc:" : “schema for storing tweets"
}
```

Avro file structure

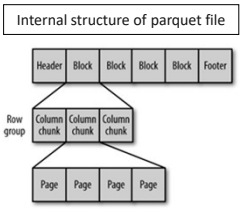


41

41

Parquet

- Column-oriented binary file format
- Efficient in terms of disk I/O when specific columns need to be queried
- Supports (page) compression and splitting
- Supports nested columns (Dremel encoding)



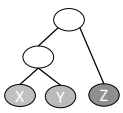
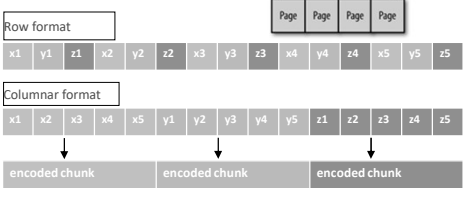


Table representation

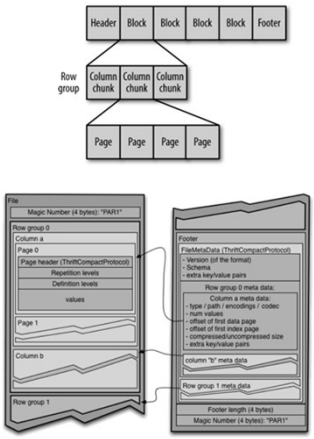
X	Y	Z
x1	y1	z1
x2	y2	z2
x3	y3	z3
x4	y4	z4
x5	Y5	z5



42

Parquet file structure & configuration

Internal structure of parquet file



Configurable parquet parameters

Property name	Default value	Description
parquet.block.size	128 MB	The size in bytes of a block (row group).
parquet.page.size	1MB	The size in bytes of a page.
parquet.dictionary.page.size	1MB	The maximum allowed size in bytes of a dictionary before falling back to plain encoding for a page.
parquet.enable.dictionary	true	Whether to use dictionary encoding.
parquet.compression	UNCOMPRESSED	The type of compression: UNCOMPRESSED, SNAPPY, GZIP & LZO

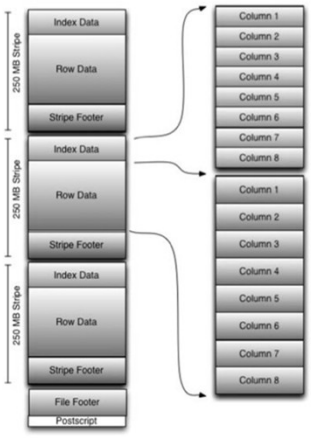
Parquet is optimized for high compression and high scan efficiency

43

43

Optimized row columnar (ORC)

- RCFile
  - Every column is compressed individually within the row group
- ORC File
  - Block-mode compression
  - Data type support
  - Ordered data store (within one stripe)
- Stores collections of rows and within the collection the row data is stored in columnar format
- Introduces a lightweight indexing that enables skipping of irrelevant blocks of rows
- Splittable: allows parallel processing of row collections
- Indices with column-level aggregated values (min, max, sum and count)



44

References

- Thanh, Tran Doan, et al. "A taxonomy and survey on distributed file systems." *2008 Fourth International Conference on Networked Computing and Advanced Information Management*. Vol. 1. IEEE, 2008.
- Levy, Eliezer, and Abraham Silberschatz. "Distributed file systems: Concepts and examples." *ACM Computing Surveys (CSUR)* 22.4 (1990): 321-374.
- Rao, T. Ramalingeswara, et al. "The big data system, components, tools, and technologies: a survey." *Knowledge and Information Systems* (2019): 1-81.
- Borthakur, Dhruba. "HDFS architecture guide." *Hadoop Apache Project* 53.1-13 (2008): 2.
- Ghemawat, Sanjay, Howard Gobioff, and Shun-Tak Leung. "The Google file system." *Proceedings of the nineteenth ACM symposium on Operating systems principles*. 2003.
- Vohra, Deepak. "Apache parquet." *Practical Hadoop Ecosystem*. Apress, Berkeley, CA, 2016. 325-335.

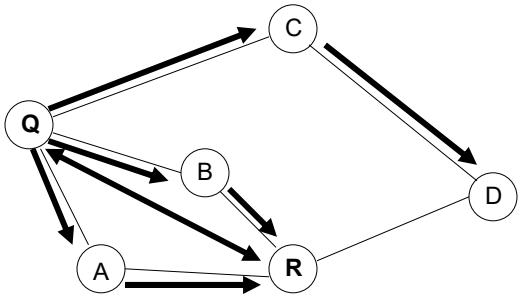
45

Q & A

46

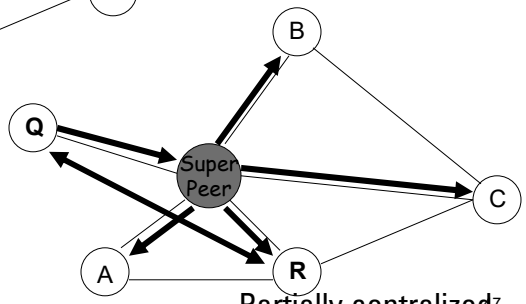
46

Peer to Peer (P2P)



Purely decentralized

- Processes name-based requests for data;
- Each node can make requests,
- Forward requests
- Return data



Partially centralized

47

47

Classifying P2P systems

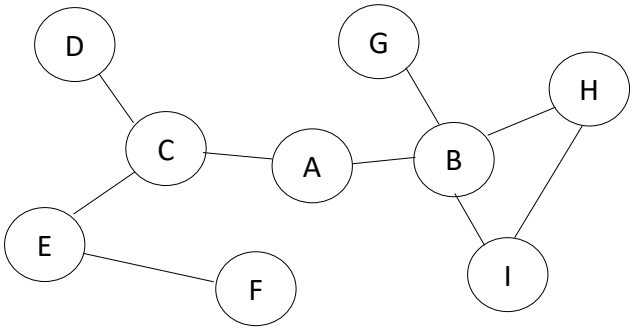
- Unstructured systems (e.g. *Napster, Gnutella*)
  - Central catalog
  - Distributed catalog, flooding technique to search
- Structured systems (e.g. *Pastry, Chord, Can, Tapestry*)
  - Distributed catalog, Distributed Hash Table Systems (DHTS)

48

48

Gnutella - Flooding technique

- Node A wants a data item; it asks B and C
  - If B and C don't have it, they ask their neighbors, etc.
  - What are the implications of this model?



49

49

Basis of structured P2P

- Requirements: building a DHT where the number of buckets stays constant, even if the number of machines changes
  - Requires a mapping from hash entries to nodes
  - Don't need to re-hash everything if node joins/leaves
  - Only the mapping (and allocation of buckets) needs to change when the number of nodes changes
- Examples: CAN, Pastry, Chord, ...

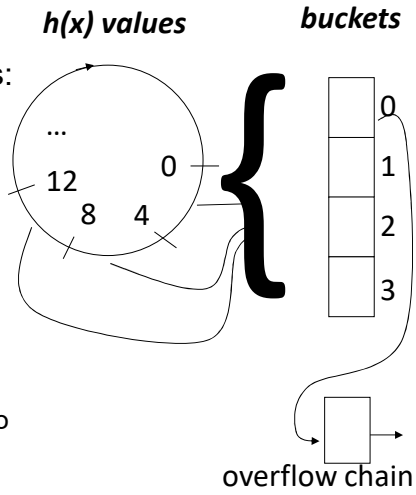
➢Consistent hashing

50

50

Hashing

- Start with a hash function with a uniform distribution of values:
  - h(name) → a value (e.g., 32-bit integer)
- Map from values to hash buckets
  - Generally using mod (# buckets)
- Put items into the buckets
  - May have "collisions" and need to chain



51

51

Dividing HT across machines

- Simple distribution – allocate some number of hash buckets to various machines
  - Can give this information to every client, or provide a central directory
  - Can evenly or unevenly distribute buckets
  - Lookup is very straightforward
- A possible issue – data skew: some ranges of values occur frequently
  - Can use *dynamic hashing* techniques
  - Can use better hash function, e.g., SHA-1 (160-bit key)

52

52

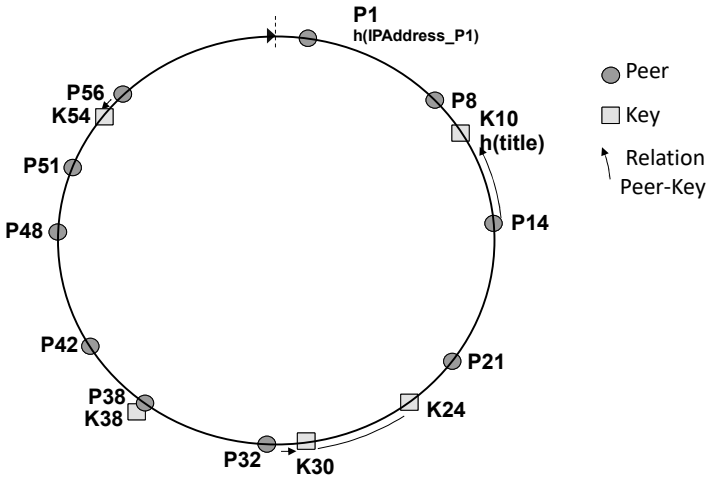
Some issues

- What if the set of servers holding the inverted index is *dynamic*?
  - Our number of buckets changes
  - How much work is required to reorganize the hash table?
- Solution: *consistent hashing*

53

53

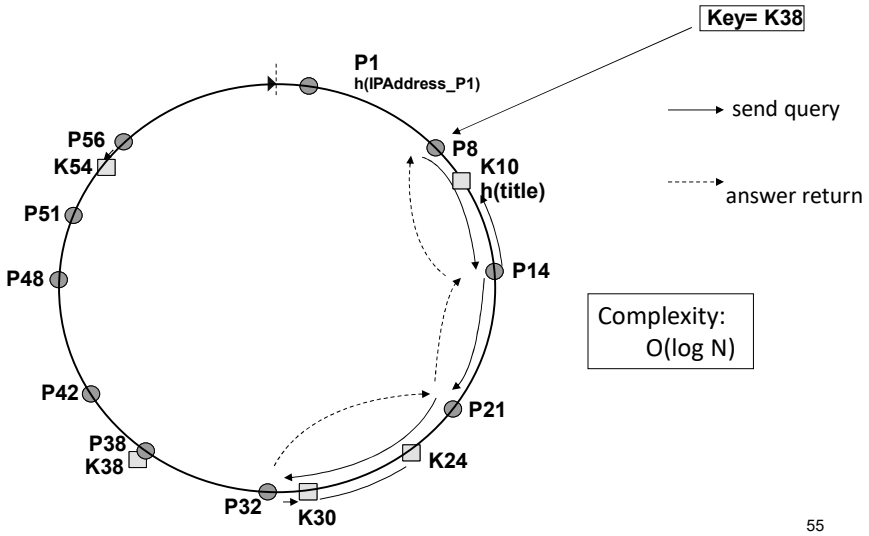
Chord: DHTs Example



54

54

Queries in *Chord*



55

55